DATA CLEANING DOCUMENTATION AND REPORT

Superstore Dataset

Week 1: Data Collection, Cleaning & Preparation

Prepared by: Khorombi Vhuthuhawe, vhuthuhawejojos\@gmail.com
Project: Data Analysis Internship
Week: 1
Tools Used: Python, Pandas, NumPy, Jupyter Notebook
Dataset: Superstore Dataset

---

1. Introduction

This report documents the data collection, inspection, cleaning, and preparation process performed on the Superstore dataset during Week 1 of the data analysis project.

The purpose of the data-cleaning process was to transform the raw dataset into a structured and analysis-ready dataset that can be used for further exploratory data analysis and business insights.

Python was used as the primary programming language, with Pandas and NumPy used for data manipulation and analysis.

The cleaning process followed a systematic approach consisting of:

1. Loading the raw dataset.
2. Inspecting the dataset structure.
3. Checking data types.
4. Converting date fields.
5. Checking for missing values.
6. Checking for duplicate records.
7. Standardising column names.
8. Checking categorical data for inconsistencies.
9. Examining numerical data.
10. Checking for potentially invalid values.
11. Detecting potential outliers.
12. Performing final data-quality checks.
13. Saving the cleaned dataset.

---

2. Dataset Collection

The Superstore dataset was provided in CSV format and loaded into Python using the Pandas `read_csv()` function.

The dataset was loaded using the following code:

```
df = pd.read_csv("superstore.csv", encoding="latin1")
```

The `latin1` encoding was specified to ensure that the CSV file could be read correctly.

The first five records were displayed using:

```
df.head()
```

This provided an initial view of the dataset and allowed the structure and values of the records to be visually inspected.

---

3. Dataset Structure Inspection

The dataset structure was examined using Pandas functions such as `shape()`, `info()`, and `columns`.

The following information was inspected:

- Number of rows
- Number of columns
- Column names
- Non-null values
- Data types

The following code was used:

```
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])
print("Dataset shape:", df.shape)

df.info()

print(df.columns.tolist())
```

This inspection provided an understanding of the size and structure of the raw dataset before any cleaning was performed.

---

4. Data Type Inspection

The data types of all columns were examined using:

```
print(df.dtypes)
```

This step was important because different variables require appropriate data types for analysis.

For example, dates should be stored as datetime values rather than text.

---

5. Date Formatting

The dataset contains `Order Date` and `Ship Date` fields.

These columns were converted to Pandas datetime format using:

```
df["Order Date"] = pd.to_datetime(df["Order Date"])

df["Ship Date"] = pd.to_datetime(df["Ship Date"])
```

The resulting data types were then checked.

Reason for Cleaning

Converting the date columns to datetime format allows the dataset to be used for future time-based analysis, such as:

- Sales by year
- Sales by month
- Order trends
- Shipping time analysis
- Seasonal trends

---

6. Missing Value Assessment

Missing values were checked for every column using:

```
missing_values = df.isnull().sum()

print("Missing values by column:")
print(missing_values)
```

The total number of missing values was also calculated:

```
total_missing = df.isnull().sum().sum()

print("Total missing values:", total_missing)
```

The percentage of missing values per column was calculated using:

```
missing_percentage = (df.isnull().sum() / len(df)) * 100

print(missing_percentage)
```

Cleaning Decision

Missing values were assessed before making any decision to remove or replace them.

The approach was to avoid automatically deleting records without first determining whether the missing values could affect the analysis.

---

7. Duplicate Record Assessment

Duplicate records were checked using:

```
duplicate_count = df.duplicated().sum()

print("Duplicate rows:", duplicate_count)
```

If duplicate records were present, they were removed using:

```
if duplicate_count > 0:
    df = df.drop_duplicates()
    print("Duplicate rows removed.")
else:
    print("No duplicate rows found.")
```

The dataset was then checked again:

```
print("Duplicate rows after cleaning:", df.duplicated().sum())
```

Cleaning Decision

Complete duplicate records were treated as unnecessary repeated observations because they could cause transactions to be counted more than once and potentially distort the analysis.

---

8. Standardisation of Column Names

The column names were standardised to make them easier to use in Python.

The following transformations were applied:

- Leading and trailing spaces were removed.
- Column names were converted to lowercase.
- Spaces were replaced with underscores.

The code used was:

```
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
```

For example:

Before:

```
Order Date
Customer Name
Product Name
```

After:

```
order_date
customer_name
product_name
```

Reason for Cleaning

Standardised column names improve readability and make it easier to reference variables consistently in Python.

---

9. Categorical Data Assessment

Categorical variables were inspected by examining their unique values.

The following variables were checked:

- Category
- Segment
- Region
- Ship Mode
- State
- City

Examples of the code used include:

```
print(df["category"].unique())

print(df["segment"].unique())

print(df["region"].unique())

print(df["ship_mode"].unique())
```

The number of unique states and cities was also examined:

```
print(df["state"].nunique())

print(df["city"].nunique())
```

Cleaning Decision

Categorical variables were inspected for possible spelling differences, unexpected categories, or inconsistent values.

This step helps ensure that categories can be grouped correctly during future analysis.

---

10. Numerical Data Assessment

The main numerical variables were examined using descriptive statistics.

The following variables were assessed:

- Sales
- Quantity
- Discount
- Profit

The following code was used:

```
df[["sales", "quantity", "discount", "profit"]].describe()
```

Minimum and maximum values were also examined:

```
print("Minimum Sales:", df["sales"].min())
print("Maximum Sales:", df["sales"].max())

print("Minimum Profit:", df["profit"].min())
print("Maximum Profit:", df["profit"].max())

print("Minimum Quantity:", df["quantity"].min())
print("Maximum Quantity:", df["quantity"].max())
```

Purpose

This provided an initial understanding of the numerical data and helped identify values that may require further investigation.

---

11. Invalid Value Assessment

Potentially invalid numerical values were checked.

The following code was used:

```
negative_sales = (df["sales"] < 0).sum()

negative_quantity = (df["quantity"] < 0).sum()

negative_profit = (df["profit"] < 0).sum()

print("Negative sales:", negative_sales)
print("Negative quantity:", negative_quantity)
print("Negative profit:", negative_profit)
```

Negative discounts were also checked:

```
negative_discount = (df["discount"] < 0).sum()

print("Negative discount:", negative_discount)
```

The discount range was examined:

```
print("Discount range:")
print(df["discount"].min(), "to", df["discount"].max())
```

Cleaning Decision

Negative profit values were not automatically removed because a negative profit can represent a legitimate business loss.

Potentially invalid values in sales, quantity, or discount would require investigation before being removed.

This approach prevents legitimate business transactions from being incorrectly classified as errors.

---

12. Outlier Detection

Potential outliers in the Sales variable were investigated using the Interquartile Range (IQR) method.

The first quartile (Q1) and third quartile (Q3) were calculated:

```
Q1 = df["sales"].quantile(0.25)

Q3 = df["sales"].quantile(0.75)
```

The IQR was then calculated:

```
IQR = Q3 - Q1
```

The lower and upper boundaries were calculated as:

```
lower_bound = Q1 - (1.5 * IQR)

upper_bound = Q3 + (1.5 * IQR)
```

Potential sales outliers were identified using:

```
sales_outliers = df[
    (df["sales"] < lower_bound) |
    (df["sales"] > upper_bound)
]
```

The number of potential outliers was then calculated:

```
print("Potential sales outliers:", len(sales_outliers))
```

Examples of potential outliers were inspected using:

```
sales_outliers[
    ["product_name", "sales", "quantity", "profit"]
].head(10)
```

Cleaning Decision

Potential outliers were identified but were not automatically removed.

A high sales value does not necessarily mean that the record is incorrect. It may represent a legitimate large transaction.

Therefore, the potential outliers were retained unless further investigation confirmed that they were errors.

---

13. Creation of Cleaned Dataset

After the cleaning and preparation process, a separate copy of the DataFrame was created:

```
cleaned_df = df.copy()
```

This created the cleaned dataset while maintaining the original DataFrame separately.

The first records of the cleaned dataset were then inspected:

```
cleaned_df.head()
```

---

14. Final Data Quality Validation

A final quality check was performed after the cleaning process.

The final dataset size was checked:

```
print("Final dataset shape:", cleaned_df.shape)
```

Missing values were checked again:

```
print("Total missing values:",
      cleaned_df.isnull().sum().sum())
```

Duplicate records were checked again:

```
print("Total duplicate rows:",
      cleaned_df.duplicated().sum())
```

The final data types were inspected:

```
print(cleaned_df.dtypes)
```

The final column names were also checked:

```
print(cleaned_df.columns.tolist())
```

Purpose

The final validation ensured that the cleaning process had produced a dataset suitable for further analysis.

---

15. Basic Dataset Metrics

Basic metrics were calculated from the cleaned dataset.

Total Sales

```
total_sales = cleaned_df["sales"].sum()

print("Total Sales:", total_sales)
```

Total Profit

```
total_profit = cleaned_df["profit"].sum()

print("Total Profit:", total_profit)
```

Total Quantity

```
total_quantity = cleaned_df["quantity"].sum()

print("Total Quantity:", total_quantity)
```

Additional unique counts were calculated for:

- Orders
- Customers
- Products

```
print("Number of Orders:",
      cleaned_df["order_id"].nunique())

print("Number of Customers:",
      cleaned_df["customer_id"].nunique())

print("Number of Products:",
      cleaned_df["product_id"].nunique())
```

These metrics provide an initial overview of the dataset and will be useful for subsequent exploratory analysis.

---

16. Cleaned Dataset Export

The cleaned dataset was exported to a new CSV file using:

```
cleaned_df.to_csv(
    "superstore_cleaned.csv",
    index=False
)
```

The cleaned dataset was saved separately from the original dataset.

File Structure

The project therefore contains:

```
superstore.csv
        ↓
Data Cleaning Process
        ↓
superstore_cleaned.csv
```

The original dataset is retained so that the cleaning process can be reproduced and compared with the cleaned version

---

17. Challenges and Considerations

Several considerations were taken into account during the cleaning process.

17.1 Avoiding Unnecessary Data Removal

Data was not removed simply because a value appeared unusual. This is important because unusual observations can represent genuine business transactions.

17.2 Negative Profit

Negative profit was treated as a potentially valid business outcome rather than automatically being classified as an error.

17.3 Outliers

The IQR method was used to identify potential sales outliers. However, the presence of an outlier does not necessarily mean that the record is incorrect.

17.4 Maintaining the Raw Dataset

The original dataset was kept unchanged while a separate cleaned dataset was created. This allows the cleaning process to remain reproducible.20. Conclusion

The Week 1 data-cleaning process successfully established a structured workflow for preparing the Superstore dataset for analysis.

The raw dataset was loaded and inspected using Python, Pandas, and NumPy. The dataset structure and data types were examined, date fields were converted into datetime format, and missing values and duplicate records were assessed.

Column names were standardized, categorical variables were inspected for consistency, and numerical variables were examined using descriptive statistics. Potentially invalid values were investigated, while potential sales outliers were identified using the IQR method.

Importantly, potential outliers and negative profit values were not automatically removed because they may represent legitimate business transactions.

After the cleaning and validation process, a separate cleaned dataset was created and exported as `superstore_cleaned.csv`.

As I am still an intern and developing my skills in data analysis, I understand that my work may require corrections, improvements, or further refinement. I would greatly appreciate any feedback, input, or suggestions from experienced analysts.

If there is anything in my report that you would like to discuss, correct, or approach differently, please feel free to contact me. I am open to constructive feedback and discussions, and I see every correction as an opportunity to learn, improve my skills, and gain a better understanding of professional data analysis practices.

I am also open to any ideas or recommendations that could improve the project. I am willing to learn and apply the guidance provided as I continue developing as a data analyst.